Chameleon: a Heterogeneous and Disaggregated Accelerator System for Retrieval-Augmented Language Models

Wenqi Jiang
Systems Group, ETH Zurich
wenqi.jiang@inf.ethz.ch

Marco Zeller
Systems Group, ETH Zurich
mzeller@student.ethz.ch

Roger Waleffe
University of Wisconsin Madison
waleffe@wisc.edu

Torsten Hoefler
SPCL, ETH Zurich
torsten.hoefler@inf.ethz.ch

Gustavo Alonso
Systems Group, ETH Zurich
alonso@inf.ethz.ch

ABSTRACT

A Retrieval-Augmented Language Model (RALM) combines a large language model (LLM) with a vector database to retrieve context-specific knowledge during text generation. This strategy facilitates impressive generation quality even with smaller models, thus reducing computational demands by orders of magnitude. To serve RALMs efficiently and flexibly, we propose *Chameleon*, a heterogeneous accelerator system integrating both LLM and vector search accelerators in a disaggregated architecture. The *heterogeneity* ensures efficient serving for both inference and retrieval, while the *disaggregation* allows independent scaling of LLM and vector search accelerators to fulfill diverse RALM requirements. Our Chameleon prototype implements vector search accelerators on FPGAs and assigns LLM inference to GPUs, with CPUs as cluster coordinators. Evaluated on various RALMs, Chameleon exhibits up to 2.16× reduction in latency and 3.18× speedup in throughput compared to the hybrid CPU-GPU architecture. The promising results pave the way for adopting heterogeneous accelerators for not only LLM inference but also vector search in future RALM systems.

PVLDB Reference Format:
Wenqi Jiang, Marco Zeller, Roger Waleffe, Torsten Hoefler, and Gustavo Alonso. Chameleon: a Heterogeneous and Disaggregated Accelerator System for Retrieval-Augmented Language Models. PVLDB, 18(1): 42 - 52, 2024.
doi:10.14778/3696435.3696439

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/fpgasystems/Chameleon-RAG-Acceleration.

1 INTRODUCTION

Vector databases facilitate the development of *Retrieval-Augmented Language Models (RALMs)*, an increasingly popular approach to serve generative large language models (LLMs). The architecture of RALM, as shown in Figure 1, allows the LLM to focus on learning linguistic structures, while incorporating context-specific knowledge during inference. Specifically, the external textual knowledge is encoded as vectors using LLMs and stored in a vector database. Given an inference context (e.g., a prompt), the knowledge retriever identifies relevant knowledge in the database via vector search, which assesses relevance by computing the similarity between the context vector and the database vectors. The retrieved texts are then incorporated into the LLM to facilitate high-quality generation.

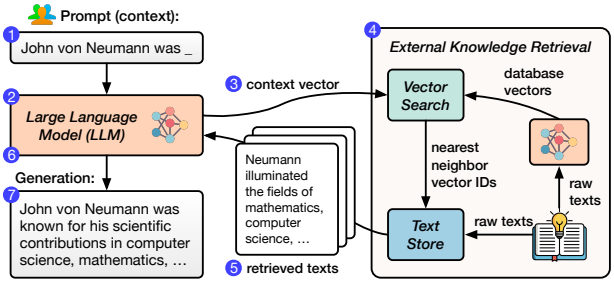

Figure 1: A retrieval-augmented language model (RALM).

RALMs show three major advantages over conventional LLMs. *First of all*, RALMs, even using smaller LLMs with one to two orders of magnitude fewer parameters, can match or surpass the generation quality of conventional LLMs on various tasks [23, 31, 41–43, 48, 49], thus significantly lowering the inference cost. This is because conventional LLMs rely on a vast number of parameters trained on massive datasets to capture and retain textual knowledge [8, 12, 67, 74], while RALMs can integrate retrieved knowledge during inference, not burdening the LLM's parameters. *Moreover*, knowledge editing in RALMs is as straightforward as updating the database, enabling efficient integration of new or private knowledge [3, 4]. In contrast, updating knowledge in conventional LLMs is inflexible, requiring additional training [7, 49]. *Finally*, RALMs enhance the reliability and interpretability of generated content by sourcing knowledge externally, while conventional LLMs are prone to producing non-factual content, known as hallucination [49, 50].

Despite its advantages, efficient RALM inference presents **two challenges**. *First, the workload characteristics of the LLM and the retriever are distinct.* While the LLM inference primarily relies on rapid tensor operations, the vector search system — often utilizing fast and memory-efficient search algorithms like Product Quantization (PQ) [35] — demands both substantial memory capacity to hold the vectors and fast processing of quantized database vectors during query time. *Second, the diverse range of RALM configurations leads to shifting system requirements and bottlenecks.* Regarding retrieval frequency, some models retrieve once per generated token [6, 42, 57], while others retrieve only once per entire sequence [31, 49]. In

Proceedings of the VLDB Endowment, Vol. 18, No. 1 ISSN 2150-8097.
doi:10.14778/3696435.3696439

terms of scale, database sizes vary from millions [24, 49] to tens of billions of vectors (92 TB) [7, 76], and model sizes range from hundreds of millions [24, 48] to tens of billions of parameters [76].

We envision a high-performance and efficient RALM system to adhere to **two key design principles** to address the two aforementioned challenges. *Firstly, RALMs should incorporate **heterogeneous accelerators**, employing not only inference accelerators such as GPUs but also vector search accelerators*, such that both RALM components are fast and efficient. *Secondly, the heterogeneous accelerators should be **disaggregated** to support diverse RALM demands efficiently*, in contrast to a monolithic approach where a fixed number of LLM and retrieval accelerators reside on the same server. The rationale is twofold: (a) performance bottlenecks shift between various RALMs of different retrieval frequencies, database sizes, and model sizes, thus requiring a case-specific optimal balance between the two types of accelerators; and (b) a huge database (e.g., with tens of TBs of vectors [7, 76]) may necessitate more retrieval accelerators than a single server can accommodate.

To materialize this vision, we propose *Chameleon*, a heterogeneous and disaggregated accelerator system for efficient, flexible, and high-performance RALM inference. Chameleon consists of three primary components. Firstly, *ChamVS* is a distributed and accelerated vector search engine. It consists of several disaggregated memory nodes, each containing a shard of quantized database vectors in DRAM, a near-memory retrieval accelerator prototyped on FPGA, and a hardware TCP/IP stack. Secondly, *ChamLM* is a multi-GPU LLM inference engine. It produces query vectors and generates texts using the retrieved information. Lastly, a CPU coordinator server orchestrates the network communication between the retrieval and LLM accelerators.

We evaluate Chameleon with various LLM architectures, model sizes, database sizes, and retrieval frequencies. For large-scale vector search, ChamVS achieves up to 23.72× latency reduction compared to the optimized CPU baselines while consuming 5.8~26.2× less energy. For RALM inference, Chameleon achieves up to 2.16× and 3.18× speedup in latency and throughput compared to the hybrid CPU-GPU architecture. We further illustrate that the optimal balance between the two types of accelerators varies significantly across different RALMs, making disaggregation essential for achieving both flexibility and high accelerator utilization rates.

The paper makes the following **contributions:**

- We present Chameleon, an efficient RALM inference system designed around two proposed principles: accelerator heterogeneity and disaggregation.
- We design and implement ChamVS, a distributed engine for large-scale vector search, which includes:
  - Near-memory accelerators for vector search, including a novel resource-efficient top-K selection architecture.
  - A GPU-based index scanner to prune search space.
- We evaluate Chameleon on various RALMs and showcase its remarkable performance and efficiency.

2 BACKGROUND AND MOTIVATION

2.1 Retrieval-Augmented Language Models

A RALM combines an LLM [16, 66, 68] with a vector database. During inference, information relevant to the current context is retrieved from the database and utilized by the LLM to predict subsequent tokens. *We classify RALMs by the content they retrieve:*

The first category of RALMs retrieves *text chunks* containing multiple tokens related to the current context [7, 31, 39, 49, 70]. During inference, the generation context, such as a user's prompt, is encoded as a query vector to retrieve context-related knowledge, i.e., text chunks in the database with similar vector representations [7, 32, 49]. The retrieved text chunks are then integrated by the LLM, leading to the generation of output tokens. When generating long sequences, however, the generated content may gradually diverge from the initially retrieved contents. Thus, instead of initiating retrieval only once at the beginning [31, 49, 72], an effective strategy is to perform multiple retrievals during text generation to improve token generation quality [70], for instance, at a regular interval of every 8~64 generated tokens [7, 39, 70].

The second category of RALMs retrieves only the *next token* of each similar context in the database [6, 42, 57]. At each step of token generation (retrieval interval is one), the last layer's hidden state serves as the query to retrieve similar contexts and the next token of each similar context [42, 57, 82]. The next token of the current context is then predicted by interpolating the next-token probability distribution predicted by the model with that of the retrieved content [41, 42].

2.2 Large-Scale Vector Search

A vector search takes a D-dimensional query vector x as input and retrieves K similar vector(s) from a database Y, populated with many D-dimensional vectors, based on metrics like L2 distances or cosine similarity. Exact K nearest neighbor (KNN) search can be prohibitively expensive for large datasets, requiring a linear scan through all database vectors. Thus, real-world vector search systems adopt approximate nearest neighbor (ANN) search that can achieve much higher system performance. The quality of an ANN search is measured by the recall at K ($R@K$), which denotes the overlap percentage between the exact K nearest neighbors and the K returned by the ANN. In the subsequent sections, we will use the terms *vector search* and *ANN search* interchangeably.

IVF-PQ, combining the inverted-file (IVF) index and product quantization (PQ), is among the most popular vector search algorithms in RALMs [7, 31, 42, 49]. Its more frequent adoption over other ANN algorithms like graph-based vector search [17, 18, 53, 55, 56, 91, 94] is primarily due to memory efficiency: RALMs can involve large databases, with reported sizes reaching up to 30 billion vectors (92 TB) [7, 76], thus the high compression ratio offered by PQ [20, 35, 40] is essential.

Inverted-File (IVF) Index. An IVF index divides a vector dataset Y into many (*nlist*) disjoint subsets, typically using clustering algorithms like K-means. Each of these subsets is termed an IVF list. At query time, the IVF index is scanned, and only a select few (*nprobe*) IVF lists whose cluster centroids are close to the query vector are scanned, such that the search space is effectively pruned.

Product Quantization (PQ). PQ reduces memory usage and computations of vector search by compressing each database vector into m-byte PQ codes. Figure 2 overviews the workflow of PQ.

Training (quantization). All database vectors are partitioned evenly into m sub-vectors ①, which possess a dimensionality of